

Язык Javascript

По материалам курса University of
Washington

<http://www.cs.washington.edu/education/courses/cse190m/07sp/index.shtml>

Краткое введение в *Javascript*

Javascript это:

1. Интерпретируемый язык. Его интерпретатор обычно встроен в браузер.
2. Основное назначение – определять «динамическое» поведение страниц при загрузке (формирование страницы перед ее открытием) и при работе пользователя со страницей (UI элементы).
3. Текст на *Javascript* может быть вложен в HTML-страницу непосредственно или находиться в отдельном файле (как CSS).
4. Похож на языки *Java* и *C#* синтаксически, но сильно отличается от них по внутреннему содержанию.

Характеристика *Javascript*

Некоторые важнейшие характеристики *Javascript* :

1. Язык объектно-ориентированного программирования. Объекты в языке имеют «тип», «атрибуты» и «методы»

```
"John, Jane, Paul, Michael".split(",").length
```

2. Переменные не имеют заранее заданного типа, то есть в разные моменты времени могут содержать значения разных типов

```
var number = 25;   number = (number < 0);   number = "25";
```

3. Типы объектов могут быть: `number`, `string`, `function`, `object`, `undefined`. Оператор `typeof` позволяет «вычислить» тип объекта.

```
typeof 25 == "number"    typeof null == "object"
```

Основные встроенные типы

Есть набор встроенных «классов», порождающих «объекты», различающиеся набором атрибутов и методов. Программисты могут динамически изменять поведение этих «классов» и создавать свои собственные. Каждый «класс» является объектом, у которого есть «прототип», определяющий набор атрибутов и методов у всех вновь создаваемых объектов этого класса.

Типы, встроенные в язык, это:

- `Number` : 64-х-разрядные числа с плавающей точкой.
- `String` : строки с символами в формате Unicode.
- `Array` : массивы с переменными границами.
- `Function` : Функции. Каждая функция, кроме того, может служить конструктором объекта.
- `Boolean`, `Date`, `Math`, `RegExp` : логические значения, даты,...

Некоторые сведения о синтаксисе

Описание переменных:

```
var count = 25,  
    msg = 'Сообщение об ошибке';  
var nullVar; // получает начальное значение null
```

Операции такие же, как в Java и C#, но более широко используется преобразование типов

+	-	*	/	%	++	--	=	+=	-=	*=
/=	%=	==	!=	>	<	>=	<=	&&		!

`2 + '3' == '23'`, но `2 + 3 == 5`

Многие операторы очень похожи на соответствующие операторы Java и C#, но могут иметь некоторые отличия в семантике.

```
for (var i = 0; i < 100; ++i) { ... }  
if (x * y < 100) { ... } else { ... }  
try { ... } catch (e) { ... } finally { ... }
```

Объекты, встроенные в браузеры

При программировании можно использовать ряд встроенных объектов. Основные из них это:

- `window` : представляет «глобальный контекст» и позволяет работать с атрибутами и методами окна.
- `document` : загруженная страница со своей структурой элементов.
- `navigator` : объект, представляющий браузер и его свойства.
- `location` : характеристики текущего URL (порт, хост и т.п.).
- объекты, представляющие элементы различных типов в HTML-странице, такие как `<body>`, `<link>`, `<img>` и т.п.
- события (events), возникающие от действий пользователя, например, нажатие кнопки мыши (`click`), загрузка новой страницы (`load`) и т.д.

Включение Javascript в HTML-страницу

Фрагменты кода можно включать в заголовок или тело HTML-документа. Кроме того, можно разместить код в отдельном файле, а в HTML-странице разместить ссылку на этот файл.

```
<html>
  <head>
    <script type="text/javascript"> ... </script>
    <script type="text/javascript" src="scripts/myscript1.js"/>
  </head>
  <body>
    <script type="text/javascript"> ... </script>
    <script type="text/javascript" src="scripts/myscript2.js"/>
  </body>
</html>
```

Код, ссылки на который размещены в заголовке, просто подсоединяется к странице и может быть использован, например, для определения реакций на пользовательские события.

Код, ссылки на который размещены в теле, выполняется при загрузке страницы и может непосредственно использоваться для формирования содержания страницы во время загрузки.

Два простых примера

Метод `document.write` используется для непосредственного включения HTML-текста в содержимое страницы, например, можно сгенерировать длинный текст в параграфе:

```
<body>
  <p>
    <script type="text/javascript">
      for (var i = 0; i < 100; ++i) {
        document.write("Hello, world! ");
      }
    </script>
  </p>
</body>
```

для лекции `javascript\helloworld.html`

Два простых примера (продолжение)

Во втором примере датчик случайных чисел используется для генерации случайной ссылки (из заданного набора):

```
<body>
  <p>
    <script type="text/javascript">
      var rand = Math.random();           // в диапазоне: [0, 1)
      var numb = Math.floor(rand * 10);
      var image = "images/image" + numb + ".jpg";
      var insert = "<img class=\"floatRight\" src=\"\" +
                    image + \"\" alt=\"фотография цветочка\"/>";
      document.write(insert);
    </script>
  </p>
</body>
```

[для лекции javascript\randomPicture.html](#)

Тип String

Строки заключаются либо в апострофы, либо в двойные кавычки

```
var slogan = "Don't be evil!";  
var image = '';
```

Операции над строками: + < > == !=

"2" + "3"	"23"	"a" == "A"	false
"10" < "5"	true	5 == "5"	true
10 < "5"	false	5 == = "5"	false
5 + "5"	"55"		

Атрибут строки: length – длина строки.

```
"abc".length == 3
```

Преобразования типов: String(n) Number(s)

```
string(10) < "5" == true      Number('3.' + '14') == 3.14
```

Стандартные методы объектов типа String

charAt, indexOf, lastIndexOf, replace, split,
substr, substring, toLowerCase, toUpperCase

Примеры:

"Google".charAt(3)	"g"
"Google".indexOf("o")	1
"Google".lastIndexOf("o")	2
"Google".replace("o", "oo")	"Gooogle"
"Google".replace(/o/g, "oo")	"Gooooogle"
"Google".split("o")	["G", "", "gle"]
"Google".substr(1, 3)	"oog"
"Google".substring(1, 3)	"oo"
"Google".toLowerCase()	"google"
"Google".toUpperCase()	"GOOGLE"

Тип Number

Числа – это 64-х-разрядные двоичные числа с плавающей точкой.

<code>Number.MIN_VALUE</code>	<code>5e-324</code>
<code>Number.MAX_VALUE</code>	<code>1.7976931348623157e+308</code>
<code>Number.NaN</code>	<code>NaN</code>
<code>Number.POSITIVE_INFINITY</code>	<code>Infinity</code>
<code>Number.NEGATIVE_INFINITY</code>	<code>-Infinity</code>

Операции над числами: `+` `-` `*` `/` `%` `<` `>` `==` `!=`

<code>3.14 % 2</code>	<code>1.14</code>
-----------------------	-------------------

Функции преобразования: `parseInt`, `parseFloat`, `Number`, `toString`

<code>parseInt("3.14")</code>	<code>3</code>
<code>parseFloat("*3.14")</code>	<code>NaN</code>
<code>Number("3.xaxa")</code>	<code>NaN</code>
<code>3.14.toString()</code>	<code>"3.14"</code>
<code>isNaN(3.14 / 0)</code>	<code>false</code>
<code>isNaN(0 / 0)</code>	<code>true</code>

Тип Boolean

Стандартные логические значения – `true` и `false`. Однако в качестве условий можно использовать любое значение.

"Истинные" условия:

```
if (2 < 5)
```

```
if (25)
```

```
if ('Google могуч и ужасен')
```

"Ложные" условия:

```
if ("")
```

```
if (0)
```

```
if (null)
```

Логические условия используются в условных операторах и операторах циклов.

```
if (x < y) { z = x; } else { z = y; }
```

```
while (x < 100) { x = x * 2; n++; }
```

```
do { x = Math.floor(x / 2); n++; } while (x > 0);
```

```
for (var y = 0, x = 0; x < 100; ++x) { y += x; }
```

Тип Date

Объекты типа Date содержат дату в виде числа миллисекунд, прошедших с 1 января 1970 г. Диапазон от -10^8 до 10^8 дней от 1 января 1970 г.

Конструкторы:

```
var now = new Date();           // сейчас
var january1st1970 = new Date(0); // дата в миллисекундах
var gagarin = new Date(1961, 3, 12);
var newYear = new Date("January 1, 2009");
```

Методы, применимые для работы с датами: getDate, getMonth, getFullYear, getTime, getTimezoneOffset, setDate, setFullYear,...

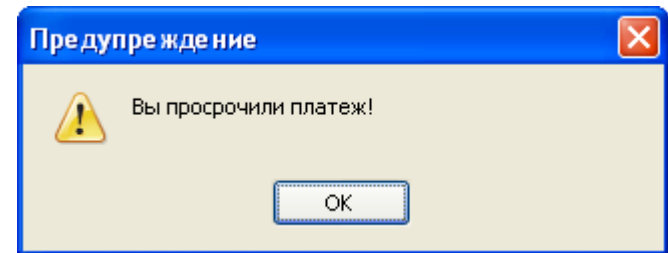
```
function DaysToDate(day, month) {
    var now = new Date(), year = now.getFullYear();
    var bd = new Date(year, month-1, day);
    var fullDay = 24 * 60 * 60 * 1000;
    var diff = Math.ceil((bd - now) / fullDay);
    return diff < 0 ? diff + 365 : diff;
}
```

[для лекции javascript\todate.html](#)

Сообщения, выдаваемые в рорир-окнах

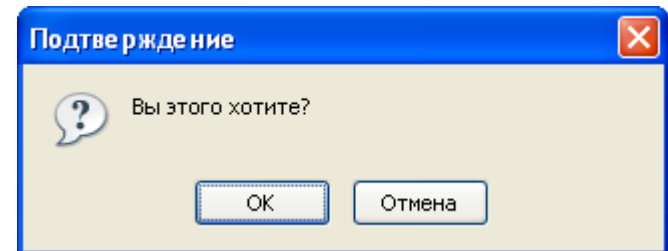
Три стандартные функции используются для генерации сообщений в рорир-окнах: `alert`, `confirm`, `prompt`.

```
alert('Вы просрочили платеж!');
```



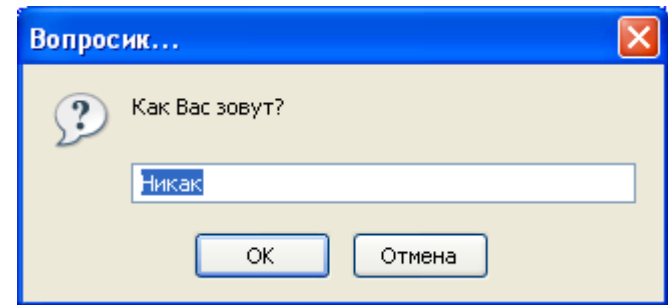
```
confirm('Вы этого хотите?');
```

Выдает **true** или **false**



```
var name = prompt('Как Вас зовут?',  
  'Никак', 'Вопросик...');
```

Выдает **введенную строку** или **null**



События и реакции на них

Имеется большое количество событий, которые можно разделить на следующие классы:

- события от мыши (click, dblclick, mousedown,...);
- события от клавиатуры (keypress, keydown,...);
- события от элементов ввода (focus, submit, select,...);
- события страницы (load, unload, error,...);

Один из способов программирования состоит в определении реакции на события непосредственно в описании элемента, например:

```
<p>День независимости России  
  <span style="color: blue; text-decoration: underline;"  
    onclick=  
      "alert('Осталось ' + DaysToDate(12, 6) + 'дней');">  
    12 июня</span>.  
</p>
```

Недостаток этого способа: javascript-текст
опять смешивается с содержанием страницы.

Тип Array

Существует несколько способов создания массива:

```
var holidays = ["1 января", "7 января", "23 февраля"];  
var holidays = new Array("1 января", "7 января", "23 февраля");  
var holidays = new Array(3);  
holidays[0] = "1 января";  
holidays[1] = "7 января";  
holidays[2] = "23 февраля";
```

Атрибут массива: `length` – длина массива.

```
var myArray = new Array();  
myArray[2] = new Date(2008,2,23);  
myArray[5] = new Date(2008,5,9);  
myArray.length == 6
```

Тип Array (продолжение)

Методы, определенные для работы с массивом:

`concat`, `join`, `pop`, `push`, `shift`, `unshift`, `slice`

```
var names = ["Петя", "Вася"];
```

```
names = names.concat(["Сереза", "Наташа"], ["Оля", "Люба"]);
```

```
names == ["Петя", "Вася", "Сереза", "Наташа", "Оля", "Люба"]
```

```
var s = names.join(';');
```

```
s == "Петя;Вася;Сереза;Наташа;Оля;Люба"
```

```
var e = names.pop();
```

```
e == "Люба"
```

```
names == ["Петя", "Вася", "Сереза", "Наташа", "Оля"]
```

```
var l = names.push("Саша");
```

```
l == 6
```

```
names == ["Петя", "Вася", "Сереза", "Наташа", "Оля", "Саша"]
```

`shift` и `unshift` – точно так же, как `pop` и `push`, но с началом массива.

```
names = names.slice(1, 4);
```

```
names == ["Вася", "Сереза", "Наташа", "Оля"]
```

Тип Array (продолжение)

Еще методы, определенные для работы с массивом:

`reverse`, `sort`, `splice`, `toString`

```
var names = ["Вася", "Сереза", "Наташа", "Оля"];
```

```
names.reverse();
```

```
names == ["Оля", "Наташа", "Сереза", "Вася"]
```

```
names.sort();
```

```
names == ["Вася", "Наташа", "Оля", "Сереза"]
```

```
var a = [5, 3, 40, 1, 10, 100].sort();
```

```
a == [1, 10, 100, 3, 40, 5]
```

```
var a = [5, 3, 40, 1, 10, 100].sort(function(a,b){return a-b;});
```

```
a == [1, 3, 5, 10, 40, 100]
```

```
names.splice(1, 2, "Саша", "Таня", "Нина");
```

```
names == ["Вася", "Саша", "Таня", "Нина", "Сереза"]
```

`toString` – точно так же, как `join(',')`.

```
names.toString() == "Вася,Саша,Таня,Нина,Сереза"
```

ФУНКЦИИ

```
<html>
```

```
<body>
```

```
Простые числа выделены красным цветом.<BR>
```

```
<style>
```

```
  .myred { color:red; }
```

```
</style>
```

```
<script>
```

```
  function prost(n)
```

```
  {
```

```
    for (i=2;i<n;i++)
```

```
      if ((n % i)==0) return 0;
```

```
    return 1;
```

```
  }
```

```
  for (papa=1;papa<=200;papa++)
```

```
    if (prost(papa)==0)
```

```
      document.writeln(papa+' ');
```

```
    else document.writeln('<span class="myred"> '+papa+  
                          '</span>');
```

```
</script>
```

```
</body>
```

Обработка событий

```
<html>
<body>

<script>
var n=0;
</script>
```

Это пример кнопки:


```
<button onClick="n=n+1;alert('Число нажатий кнопки:
'+n) ">Кнопка</button>
```

```
</body>
```

```
function knock ()
{
  n=n+1;
  alert('Число нажатий кнопки: '+n);
}
</script>
```

Это пример кнопки:


```
<button onClick="knock()"> Кнопка </button>
```

Перечислим наиболее часто используемые атрибуты тегов, отвечающие за обработку событий:

<i>onblur:</i>	потеря фокуса;
<i>onchange:</i>	изменение значения элемента формы;
<i>onclick:</i>	щелчок левой кнопкой мыши на элементе;
<i>ondblclick:</i>	двойной щелчок левой кнопкой мыши на элементе;
<i>onfocus:</i>	получение фокуса;
<i>onkeydown:</i>	клавиша нажата, но не отпущена;
<i>onkeypress:</i>	клавиша нажата и отпущена;
<i>onkeyup:</i>	клавиша отпущена;
<i>onload:</i>	документ загружен;
<i>onmousedown:</i>	нажата левая кнопка мыши;
<i>onmousemove:</i>	перемещение курсора мыши;
<i>onmouseout:</i>	курсор покидает элемент;
<i>onmouseover:</i>	курсор наводится на элемент;
<i>onmouseup:</i>	левая кнопка мыши отпущена;
<i>onreset:</i>	форма очищена;
<i>onselect:</i>	выделен текст в поле формы;
<i>onsubmit:</i>	форма отправлена;
<i>onunload:</i>	закрытие окна.

```
<html> <body>
<button onClick="more()">Кнопка</button>
<script> more(); </script>
<br>Текст<br>
<script>
    function more() { alert('Мопе!'); }
</script>
</body>
```

Пока веб-страница не загрузилась полностью, нельзя из JavaScript-кода обращаться к тем фрагментам программы, которые вставлены в веб-страницу ниже по тексту.

```
<html>
```

```
<body onload="more()">
```

Здесь какой-то текст

```
<script>function more() { alert('Мопе!'); } </script>
</body>
```

Изменение содержимого веб-страниц

Объект **document** имеет ряд полезных функций, например **getElementById(id)**, которые позволяют получить доступ к элементам веб-страницы и менять их содержание.

```
<html>

<body>

<style>
  div {
    background-color:yellow;
    padding:15px;
  }
</style>

<script>

function primer()
{
  s=document.getElementById('olga');
  s.innerHTML='Меня переехали!';
}

</script>
```

Пример изменения текста

```
<div id="olga" onmouseover="primer()"> Желтый  
прямоугольник </div>
```

```
</body>
```

```
<html>
<script>
function faktorial(n)
{
  var k=1;
  for (var i=1;i<=n;i++)k=k*i;
  return k;
}
function obrabotka()
{
n=document.getElementById("mama").value;
document.getElementById("otvet").value = faktorial(n);
}
</script>
<body>
Число: <input id="mama" onChange="obrabotka()" "
type="text" value="1"></input> <br>
Факториал: <input id="otvet" type="text"
value="1"></input>
</body>
```

Отслеживание координат мыши

```
<html>
<body>

<script>
    function khai(eve)
    {
        s=document.getElementById('taras');
        s.innerHTML='X='+eve.pageX+' Y='+eve.pageY;
    }

    document.captureEvents(Event.MOUSEMOVE);
    document.onmousemove=khai;
</script>

<div id="taras"></div>
</body>
```